

路线 2：约束处理、编码与罚函数调参

约束处理是量子优化里最容易被低估的一层。很多 demo 看起来在优化，其实只是找到了一个低能量但非法的 bitstring。

这条路线的核心问题是：求解器只看能量，我们如何让“合法”在能量地形里变得重要？

为什么约束是核心风险

还是看模型选择例子：

$$\begin{aligned} &\text{maximize } 9x_a + 6x_b + 7x_c \\ &\text{subject to } x_a + x_b + x_c = 1 \end{aligned}$$

如果没有约束，最优是：

$$x_a = x_b = x_c = 1$$

收益 22，很诱人，但非法。

QUBO 求解器并不知道“只能选一个”是业务铁律。它只知道能量高低。所以我们必须把约束翻译成求解器能感知的东西。

五种处理方式

约束处理大致有五条路：

1. 罚函数 penalty
2. 显式编码 encoding
3. 不等式 slack / unbalanced penalty
4. 求解后 feasibility check + repair
5. 约束保持 mixer

前四个属于这条路线。第五个会在路线 5 里单独讲。

等式约束：平方罚项

等式约束：

$$g(x) = 0$$

常用 penalty：

$$A * g(x)^2$$

Exactly-one:

$$A \cdot (x_a + x_b + x_c - 1)^2$$

Cardinality:

$$A \cdot \left(\sum_i x_i - k\right)^2$$

这种方法简单、通用、容易解释。缺点是：展开后会产生很多二次耦合，而且 penalty 权重不好选。

At-most-one: 互斥罚项

At-most-one 是:

$$x_a + x_b + x_c \leq 1$$

可以直接罚两两同时选择:

$$A \cdot (x_a x_b + x_a x_c + x_b x_c)$$

只要两个变量同时为 1，就会罚。这个形式比引入 slack 更轻。

比赛里遇到互斥选择，先想 pairwise penalty，别一上来就上大而全的 inequality 编码。

不等式约束: slack 的直觉

预算约束:

$$5x_a + 3x_b + 4x_c + 2x_x + x_y \leq 6$$

可以加入 slack 变量 $s \geq 0$ ，改成等式:

$$5x_a + 3x_b + 4x_c + 2x_x + x_y + s = 6$$

然后罚:

$$A \cdot (5x_a + 3x_b + 4x_c + 2x_x + x_y + s - 6)^2$$

slack 的问题是变量会增加。预算上限越大，slack 编码越重。很多实际问题卡住，不是因为目标函数复杂，而是因为 slack 变量把 QUBO 撑爆了。

Unbalanced penalty 的直觉

不等式只关心一边:

$$lhs \leq rhs$$

如果 `lhs` 小于 `rhs`，其实不用罚；只有超过才罚。

普通平方罚项把等式化后，会让“没用完预算”也产生结构压力。Unbalanced penalization 的目标是更偏向惩罚违反方向，减少 slack 依赖，让不等式处理更轻。

你不需要第一遍记完整公式，先记住它解决的痛点：

用更少变量处理 `inequality`，尤其是容量、预算、风险上限类约束。

比赛时如果发现 slack 变量太多，unbalanced penalty 是一个很好的改进点。

Encoding：把合法性写进变量结构

有些约束可以通过变量结构自然满足。

One-hot 编码：

$$z_0 + z_1 + z_2 + z_3 = 1$$

用来表示四选一。

Domain-wall 编码：

用一串有顺序的 bit 表示多值变量。它通常比 one-hot 少变量，也比普通 binary 更有局部结构，适合某些退火器。

Encoding 的选择很像“设计棋盘”。棋盘设计得好，算法移动起来就顺；棋盘设计得差，后面再强的求解器也吃亏。

Feasibility checker

不管前面用了什么 penalty，最后都必须检查：

这个 `bitstring` 解码后是否满足原始约束？

Feasibility checker 至少要输出：

- 每条约束是否满足
- 违反程度是多少
- 原始目标值是多少
- QUBO energy 是多少
- 是否需要 repair

排序时不要只按 raw energy。更稳的规则是：

先可行，再目标好；不可行解只能作为 `repair` 候选。

Repair: 让非法解回到合法世界

Repair 是比赛里特别实用的一招。
比如预算超了:

当前选择收益高, 但资源超 3

可以按某种规则删掉变量:

- 删收益最低的
- 删单位资源收益最低的
- 删对协同收益贡献最小的
- 用局部搜索删改组合

Repair 不丢人。真实优化系统里, 启发式求解 + repair + local search 很常见。关键是要诚实记录: 原始采样是否可行, repair 后目标是多少。

Penalty 权重怎么选

一个朴素原则:

违反硬约束得到的最大收益, 必须小于违反约束付出的 penalty。

例如多选一个模型最多多赚 9 分, 那 exactly-one penalty 至少要超过这个量级。
更实际的做法是 penalty sweep:

$$A = 1, 5, 10, 20, 50, 100$$

记录:

- feasible ratio
- best feasible objective
- best raw energy 是否可行
- 求解时间
- 系数范围

比赛展示时, penalty sweep 是很有说服力的图。它说明你不是随便拍一个罚系数。

比赛时能改哪些地方

约束分级

硬约束必须满足，软约束可以违反但要付成本。别把所有规则都当同一种 penalty。

Penalty 形式

等式用平方，不等式可以 slack、unbalanced、repair 或组合策略。

编码方式

多值变量选 binary、one-hot、domain-wall，会直接影响变量数和可行解比例。

Repair 策略

同一个 infeasible sample，不同 repair 会得到不同质量的可行解。这里很适合做算法改进。

排序规则

最终结果要按原始目标和可行性排序，而不是按 QUBO energy。

常见坑

Penalty 太小。

求解器会选择非法高收益解。

Penalty 太大。

能量地形被约束项支配，目标函数细节消失，采样器很难区分好解和一般解。

只检查总 violation。

有些约束比另一些更重要。总 violation 小，不代表业务可接受。

把 repair 当作免费。

Repair 会改变解，可能损失目标值。要记录 repair 前后。

把不等式都转成 slack。

这常常导致变量暴涨。先判断有没有更轻的处理方式。

和其他路线的关系

路线 1 给出 QUBO 语言。

路线 2 决定 QUBO 里的约束质量。

路线 3 和路线 4 都依赖这套约束处理，否则会输出不可行样本。

路线 5 会把一部分约束从 penalty 转移到 mixer 结构里。

路线 6 会把一部分复杂约束留给经典 solver 或 repair，而不是强行全塞进 QUBO。

共读任务

拿 sample_problem.json 做三件事：

1. 写出 choose_one_model 的 penalty。
2. 想一想 resource_budget 用 slack 会增加多少变量。
3. 设计一个预算超限 repair：如果超预算，你删哪个变量，为什么？

这三个问题能帮助你把“约束不是附属品”这件事真正记住。

延伸阅读

- `literature/requirements/02_constraint_handling_requirements.md`
- Hen & Spedalieri, constrained optimization with quantum annealing
- Montanez-Barrera et al., unbalanced penalization for inequalities
- Chancellor, domain-wall encoding